01 - Visión: ¿Qué son los MCPs y por qué los necesitamos?

¿Qué es MCP (Model Context Protocol)?

MCP es un protocolo estándar de Anthropic que permite a Claude (o cualquier LLM) comunicarse con herramientas/servicios externas de forma estructurada.

Analógía Simple

```
Sin MCP:
  Claude: "Te recomiendo usar this Unity script"
  Tú: Copias manualmente el código
  Tú: Lo pegas en Assets/Scripts/
  Tú: Configuras GameObjects manualmente
  = Lento, propenso a errores

Con MCP:
  Claude: "Voy a crear el script directamente"
  MCP (Unity): Crea el archivo automáticamente
  Claude: "Ahora configuro el Rigidbody"
  MCP (Unity): Modifica el prefab automáticamente
  = Rápido, sin fricción, sin copiar-pegar
```

¿Por qué dos MCPs? (Unity + Blender)

MCP Unity

Para: Desarrollo C# dentro de Unity

Casos de uso:

```
1. Crear scripts automáticamente
   Prompt: "Crea un PlayerController con movimiento WASD"
   MCP: Genera Assets/Scripts/Player/PlayerController.cs

2. Configurar GameObjects
   Prompt: "En el prefab Player, configura Rigidbody: mass=80, drag=0.3"
   MCP: Modifica el prefab directamente

3. Gestionar prefabs
   Prompt: "Crea un prefab para el Boss_01 basado en este GameObject"
   MCP: Genera Prefabs/Bosses/Boss_01.prefab

4. Compilación y debugging
```

```
   Prompt: "Compila el proyecto y dime qué errores hay"
   MCP: Ejecuta compilación, reporta errores

5. Generar ScriptableObjects
   Prompt: "Crea un SO de configuración para el Boss_01 con estos datos"
   MCP: Genera Assets/Settings/Bosses/Boss_01.asset
```

MCP Blender

Para: Modelado 3D y animación en Blender

Casos de uso:

```
1. Crear modelos 3D
   Prompt: "Crea un humanoide básico estilo Spider-Man"
   MCP: Genera modelo en Blender

2. Rigging automático
   Prompt: "Agrega un rig humano estándar Rigify"
   MCP: Crea armadura y bones automáticamente

3. Crear animaciones
   Prompt: "Crea animación de punch (20 frames)"
   MCP: Genera animación con keyframes

4. Exportación automática
   Prompt: "Exporta como FBX optimizado a Assets/Models/Characters/"
   MCP: Exporta con configuración correcta

5. Batch processing
   Prompt: "Exporta todos los modelos en carpeta X a FBX"
   MCP: Exporta múltiples assets automáticamente
```

Arquitectura General (Simplificada)

```
┌──────────────────────┐
│   Claude Code        │
│   (Este IDE)         │
└──────────────────────┘
           │
           │
           │ MCP Protocol (JSON)
           │
           │
    ┌──────┴──────┐
    │             │
    ▼             ▼
┌──────────┐  ┌──────────┐
│ MCP      │  │ MCP      │
│ Unity    │  │ Blender  │
```

```
  │ Server  │   │ Server   │
  └─────────┘   └──────────┘
       │             │
       │             │
       │ WebSocket   │ CLI
       │             │
       ▼             ▼
   ┌─────────┐   ┌─────────┐
   │ Unity   │   │ Blender│
   │ Editor  │   │ (batch)│
   └─────────┘   └─────────┘
```

Ventajas Clave

1. **Velocidad de Desarrollo**

```
Manual: Crear script → Copiar → Pegar → Configurar = 10 min
Con MCP: Descripción en texto → MCP hace todo = 30 seg
```

2. **Menos Errores**

```
Manual: Posibilidad de copiar código incorrectamente
Con MCP: El código es generado/modificado programáticamente, sin errores de copia
```

3. **Iteración Rápida**

```
Cambio de 3 personajes: Descripción × 3 = 2 minutos
vs. Hacerlo manualmente = 1-2 horas
```

4. **Automatización Compleja**

```
Crear personaje:
  1. Modelar en Blender ✓
  2. Crear rig ✓
  3. Exportar a FBX ✓
  4. Importar en Unity ✓
  5. Crear prefab ✓
  6. Configurar animator ✓

TODO en una sesión de Claude, con un prompt descriptivo
```

5. **Documentación viva**

```
Cada tarea que Claude hace es automáticamente documentada
(el código generado es el spec)
```

Flujo de Trabajo Ideal (Con MCPs)

Escenario: Crear un Boss

Sin MCPs:

1. Diseñar en documento
2. Modelar en Blender (30 min)
3. Exportar FBX manualmente
4. Importar en Unity
5. Crear prefab manualmente
6. Crear script C# para IA
7. Configurar componentes
8. Testing y debug **Total: 2-3 horas**

Con MCPs:

```
Yo: "Necesito un boss llamado Symbiote_01. Proporciono specs..."

Claude:
  1. Crea modelo en Blender (MCP Blender)
  2. Agrega rig automático (MCP Blender)
  3. Exporta a FBX (MCP Blender)
  4. Importa en Unity (MCP Unity)
  5. Crea prefab (MCP Unity)
  6. Genera script de IA Boss (MCP Unity)
  7. Configura Collider, Rigidbody, animator (MCP Unity)
  8. Compila y reporta estado

Yo: Verifico visualmente, hago ajustes finos si es necesario

Total: 15-30 minutos (INCLUYENDO testing)
```

Flujo Actual vs. Futuro

Hoy (Sin MCPs)

```
Claude genera sugerencias de código
├─ Yo copio el código
├─ Yo lo pego en Unity/Blender
├─ Yo configuro manualmente
└─ Yo reporto si algo falla
```

Mañana (Con MCPs)

```
Claude ejecuta acciones directamente
├─ MCP crea archivos automáticamente
├─ MCP configura componentes
├─ MCP compila/exporta automáticamente
└─ MCP reporta status en tiempo real
```

Limitaciones Honestas

¿Qué los MCPs NO pueden hacer?

- ❌ Decisiones artísticas complejas (qué se vea bien es subjetivo)
- ❌ Crear assets de audio automáticamente
- ❌ Substituir testing manual completamente
- ❌ Generar modelos de forma procedimental con calidad AAA

¿Dónde brillan?

- ☑ Automatizar tareas **repetitivas** (crear 5 scripts similares)
- ☑ Generar **boilerplate** rápidamente
- ☑ **Iterar rápidamente** basado en specs
- ☑ **Batch processing** (exportar 20 assets)
- ☑ **Mantener consistencia** (todos los scripts siguen el mismo patrón)

Casos de Uso Reales para ZoremGame

Crear los 4 personajes

```
Prompt: "Crea 4 personajes:
  1. Brute - humanoid fuerte
  2. Agile - humanoid delgado
  3. Mage - humanoid con aura mágica
  4. Hybrid - mezcla de brute y mage"

MCP Blender: Genera 4 modelos con variaciones
MCP Unity: Crea 4 prefabs con animadores configurados
Resultado: 4 personajes listos en 30 min
```

Crear 4 Bosses

```
Prompt: "Crea 4 bosses, cada uno diferente:
  1. Boss_01: Agresivo, patrón ataque rápido
```

```
    2. Boss_02: Defensivo, requiere estrategia
    3. Boss_03: Mágico, dispara proyectiles
    4. Boss_04: Híbrido, combina los anteriores"

  MCP Blender: Genera 4 modelos de boss
  MCP Unity: Crea 4 prefabs con scripts de IA
  Resultado: 4 bosses completamente diferentes, listos
```

Crear 4 Mapas

```
  Prompt: "Crea layouts para 4 mapas diferentes..."
  MCP: Genera layouts básicos, props, zona spawneable
  Resultado: 4 mapas esqueletales listos para detalles
```

Por qué Implementar Esto Ahora

Opción A: Sin MCPs (Enfoque tradicional)

```
  Desarrollo = 6-12 meses
  Proceso = Manual, repetitivo, lento
  Frustración = Alta (copiar-pegar todos los días)
```

Opción B: Con MCPs (Este proyecto)

```
  Setup de MCPs = 1-2 semanas
  Desarrollo = 3-6 meses (2-4x más rápido después)
  Proceso = Automatizado, repetible, fluido
  Frustración = Baja (focuses en game design, no busywork)
```

ROI: 1-2 semanas de setup = meses ahorrados después.

Siguiente: Arquitectura

Ahora que entiendes la visión, lee **02_Arquitectura.md** para entender cómo se comunican todo los componentes.